

Deployment & Usage

How to run, test, and integrate the DRCS engine.

Environment Setup

Prerequisites

- Node.js 18+
- npm or yarn
- SQLite (dev/test) or PostgreSQL (production)

Installation

```
git clone https://github.com/TellDontaMatter/dracs-engine.git
cd dracs-engine
npm install
```

Environment Variables

Create a `.env` file in the project root:

```
# Database
DATABASE_URL="file:./dev.db" # SQLite for dev
# DATABASE_URL="postgresql://user:pass@host:5432/dracs" # PostgreSQL for prod

# LLM API (required for C5 and evaluatePrompt)
ABACUS_API_KEY="your-abacus-api-key-here"

# Server
PORT=3000

# Default tenant (optional, defaults to 'zilly')
DRCS_TENANT="zilly"
```

Database Setup

Development (SQLite)

```
# Generate Prisma client
npx prisma generate

# Apply migrations
npx prisma migrate dev

# Seed Zilly tenant data
npm run seed:zilly
```

Production (PostgreSQL)

1. Update `DATABASE_URL` in `.env` to your PostgreSQL connection string
2. Change `provider` in `prisma/schema.prisma` from `sqlite` to `postgresql`
3. Run migrations:

```
npx prisma migrate deploy
```

Running the Engine

Web Console (Human Interface)

```
npm run serve
# Opens at http://localhost:3000
# Single-prompt form: type idea → get file path + caption
```

CLI (Command Line)

```
# Natural language prompt
npm run evaluate -- --prompt "make a hype post about hitting 10k followers"

# Structured input (advanced)
npm run evaluate -- \
  --tenant zilly \
  --condition "Victory celebration" \
  --caption "We did it!"

# Dry run (no deployment logging)
npm run evaluate -- --prompt "test idea" --dry-run
```

Programmatic (Code Integration)

```
import { evaluatePrompt } from 'dracs-engine';

const verdict = await evaluatePrompt('zilly', 'make a hype post about hitting 10k fol-
lowers');

console.log(verdict.decision);      // "PUBLISH"
console.log(verdict.file_path);    // "media/assets/zilly_mascot.png"
console.log(verdict.caption);      // "Victory"
console.log(verdict.source);       // "AS_IS" | "RECAPTIONED" | "GENERATED"
```

Testing

Run All Tests

```
npm test
# Runs 107 tests across 10 suites
# Uses in-memory SQLite (test.db)
# All LLM calls mocked
```

Run Specific Test Suite

```
npm test -- tests/c4.test.ts
npm test -- tests/orchestrator.test.ts
```

Test Coverage

- **Gates C1-C6:** Unit tests verify all acceptance criteria
 - **Orchestrator:** Integration tests verify gate sequence + short-circuiting
 - **Persistence:** Tenant isolation, idempotency, data integrity
 - **Assets:** Content retrieval, null file_path handling
 - **LLM:** Mocked in all tests (no real API calls in CI)
-

Multi-Tenant Configuration

Adding a New Tenant

1. Seed tenant config:

```
import { prisma } from './src/persistence/client';

await prisma.tenantConfig.create({
  data: {
    tenant_id: 'new-tenant',
    max_repetition_count: 3,
    rolling_window_days: 30,
  },
});
```

1. Seed category schema:

```
await prisma.categorySchema.create({
  data: {
    tenant_id: 'new-tenant',
    category_id: 'victory',
    name: 'Victory',
    asset_list: ['asset_001'],
    protected_flag: false,
    prestocked_flag: true,
  },
});
```

1. Seed assets:

```
await prisma.assetRegistry.create({
  data: {
    tenant_id: 'new-tenant',
    asset_id: 'asset_001',
    tag: 'canonical',
    category: 'victory',
    caption: 'We made it!',
    file_path: 'media/new-tenant/victory.png',
    content_hash: 'sha256:...',
    sealed_hash: 'sha256:...',
  },
});
```

No code changes required. All tenant behavior is driven by these three tables.

API Reference

evaluatePrompt(tenant_id, prompt)

Simplest entry point. Natural language → verdict.

Input:

- `tenant_id: string` — which tenant's config/categories/assets to use
- `prompt: string` — user's natural language idea

Output: `EvaluationVerdict` — see structure below.

Example:

```
const v = await evaluatePrompt('zilly', 'create chaos content');
// Returns: file_path, caption, decision, trail
```

evaluate(request, tenant_id)

Structured entry point. Full control over trigger/signal/need.

Input:

```

{
  trigger: {
    condition: string,
    confidence_tag?: 'strong' | 'weak',
    stakes?: 'high' | 'medium' | 'low',
    allowed_to_acknowledge?: string[],
    must_not_presume?: string[]
  },
  condition_signal: {
    category_id?: string,
    situation?: string
  },
  content_need: {
    caption: string,
    exclude_asset_ids?: string[],
    required_tag?: 'canonical' | 'recaption' | 'variation'
  },
  content_type?: string,
  deployment_context?: string
}

```

Output: EvaluationVerdict

Verdict Structure

```

{
  tenant_id: string,
  asset_id: string | null,
  caption: string | null,
  file_path: string | null,
  source: 'AS_IS' | 'RECAPTIONED' | 'GENERATED' | null,
  asset_recommendation: string | null,
  resolution_step: 'EXISTING_AS_IS' | 'RECAPTION' | 'ESCALATE_TO_C5' | null,
  governance_record_id: string,
  committed: boolean,
  deployment_id: string | null,
  trail: TrailEntry[], // gate-by-gate audit
  decision: 'PUBLISH' | 'BLOCKED',
  outcome: EvaluationOutcome,
  stopped_at_gate: GateId | null,
  reason: string
}

```

Production Checklist

Before deploying to production:

- [] Update DATABASE_URL to PostgreSQL connection string
- [] Change provider in prisma/schema.prisma to postgresql
- [] Run npx prisma migrate deploy
- [] Set ABACUS_API_KEY in production environment
- [] Seed production tenant data (categories, assets, config)

- [] Verify tenant isolation (run tests with `NODE_ENV=production`)
- [] Set up monitoring for LLM errors (C5 `ESCALATE_FAILED` outcomes)
- [] Configure usage log retention policy (C3 relies on 30-day rolling window)
- [] Review governance records regularly (C6 creates one for every trigger)

Monitoring & Observability

Key Metrics to Track

Metric	Why It Matters
C5 <code>ESCALATE_FAILED</code> rate	LLM unavailable or response empty — affects UX
C3 blocks (repetition)	Asset frequency cap hit — may need new content
C6 <code>REJECT</code> / <code>HOLD</code> disposition rate	Governance blocking ideas — review policy
Verdict outcome distribution	<code>PUBLISHED</code> vs. <code>BLOCKED</code> vs. <code>GENERATED</code> — content health
Average latency (<code>evaluatePrompt</code>)	Includes LLM call — typically ~2s

Audit Trail

Every call to `evaluate()` or `evaluatePrompt()` returns a `trail` array with one entry per gate:

```
trail: [  
  {  
    gate: 'C6',  
    name: 'Message-Idea Governance',  
    passed: true,  
    summary: 'Disposition: PUBLISH',  
    data: { disposition: 'PUBLISH', record: {...} }  
  },  
  {  
    gate: 'C2',  
    name: 'Situational Bank',  
    passed: true,  
    summary: 'Resolved category "victory"',  
    data: { category_id: 'victory', available_assets: [...] }  
  },  
  // ... C4, C5, C3  
]
```

Log this trail for every production request. It's the complete history of why the engine made its decision.

Troubleshooting

“LLM returned HTTP 401”

- **Cause:** `ABACUS_API_KEY` missing or invalid
- **Fix:** Verify `.env` has the correct key; check key hasn't expired

“No category matches situation X for tenant Y”

- **Cause:** C2 couldn't resolve the category
- **Fix:** Seed the category in `CategorySchema` for that tenant, OR the LLM picked a category name that doesn't exist

“`file_path` is null but I seeded it”

- **Cause:** Asset exists but `file_path` column not populated
- **Fix:** Run `assetRegistry.updateFilePath(tenant_id, asset_id, path)` or re-run seed with updated data

“Tests fail with ‘Cannot find module @prisma/client’”

- **Cause:** Prisma client not generated
- **Fix:** Run `npm run prisma generate`

“C3 blocks every deployment with `failed_closed: true`”

- **Cause:** Usage log integrity issue (missing/corrupted records)
- **Fix:** Inspect `UsageLog` table; verify `deployed_at` timestamps are valid; clear corrupt entries

Best Practices

1. **Always use tenant-scoped queries.** Never query `AssetRegistry` or `CategorySchema` without a `tenant_id` filter.
2. **Validate LLM output.** C5's `parseGeneration()` is defensive, but monitor for garbage responses.
3. **Log governance records.** Every C6 decision creates a record — use it for policy review.
4. **Respect the 3-step ladder.** Don't bypass C4 to call C5 directly — you'll lose caption-first reuse logic.
5. **Test with `--dry-run`.** Prevents logging deployments during testing.
6. **Keep seeds idempotent.** Zilly seed uses upsert logic — safe to re-run.

For integration questions, see: `04-reference/handoff-capsule.md`